

DATABASE MANAGEMENT SYSTEM

MCA 104

Unit-8

Internet Databases –World Wide Web, Client Side Scripting and Applets

Learning Objectives

After studying this unit, the student will be able to:

- Understand the concept of Internet Databases and their significance in storing and managing vast amounts of information accessible through the World Wide Web (WWW).
- Comprehend the role of Client Side Scripting and Applets, and how they enable dynamic content generation and interactivity within web browsers, enhancing user experiences.
- Gain knowledge of Web Servers and Sessions, and their importance in facilitating seamless communication between clients and servers, enabling personalized interactions and data storage.

Structure of the Unit

Introduction

8.1 Internet Database

8.2 World Wide Web

8.3 Client Side Scripting and Applets

8.4 Web Servers and Sessions

8.5 Web Services

8.6 Server-Side Scripting

8.7 Summary

8.8 Review Questions

8.9 Keywords

8.10 References

Introduction

The World Wide Web (WWW) is an interconnected network of web pages and resources that are accessible through the internet. The web pages are written in HTML (Hypertext Markup Language) and are linked together through hyperlinks, allowing users to navigate from one page to another. The web is a vast repository of information, services, and resources, and it relies on internet databases to store and retrieve data.

Client-Side Scripting and Applets:

- Client-side scripting refers to the execution of scripts or code on the user's web browser, directly on the client-side. Common client-side scripting languages are JavaScript, CSS (Cascading Style Sheets), and HTML. These scripts provide dynamic and interactive features on web pages, such as form validation, animations, and user interface enhancements.
- Applets, on the other hand, are small applications or programs that are embedded within web pages and run on the client's browser. Java Applets were popular in the past for providing dynamic content and interactivity. However, their usage has decreased in favor of other technologies like JavaScript due to security concerns and browser compatibility issues.

Web Servers and Sessions:

- A web server is a computer program or software that serves web pages and resources in response to requests from client browsers. When a

user enters a URL or clicks on a link, the browser sends a request to the web server, which processes the request and returns the corresponding web page to the client's browser.

- Sessions are used to maintain stateful interactions between the web server and the client's browser. When a user visits a website, a unique session identifier is created and stored as a cookie or in the URL. This identifier is used to keep track of the user's interactions and data during their visit to the website.

Services:

- Internet databases provide various services to users and applications. These services include:
 - Data Storage: Internet databases store vast amounts of data that can be accessed and retrieved by users or applications.
 - Data Retrieval: Users can search and retrieve specific data from internet databases using search engines or web applications.
 - Data Manipulation: Internet databases allow users to add, modify, and delete data through web forms and applications.
 - Authentication and Security: Databases often provide authentication and security mechanisms to control access to sensitive information.

Server-Side Scripting:

- Server-side scripting refers to the execution of scripts or code on the web server, rather than the client's browser. Common server-side scripting languages are PHP, Python, Ruby, and ASP.NET. Server-side

scripts process user input, interact with databases and generate dynamic content that is then sent to the client's browser for display.

- Server-side scripting allows for more secure and controlled processing of sensitive data and logic, as well as dynamic generation of web pages based on user input or database interactions.

8.1 Internet Database

An Internet database in the context of DBMS (Database Management System) refers to a database that is designed to be accessed and utilized over the Internet. It enables users to store, manage, and retrieve data through web applications and services. Internet databases play a crucial role in modern web-based applications, powering various online services and e-commerce platforms.

Fundamental elements of Internet databases:

- *Client-Server Architecture:* Internet databases typically follow a client-server architecture, where the database server hosts the actual database and manages data storage and retrieval, while clients (web browsers or applications) interact with the server over the internet to perform CRUD operations (Create, Read, Update, Delete).
- *Web-based Interfaces:* Internet databases use web-based interfaces to allow users to access and interact with the data. These interfaces can be web applications, websites, or APIs (Application Programming

Interfaces) that enable programmatic access to the database from various client devices.

- *Connectivity:* Internet databases rely on Internet connectivity to establish communication between clients and the database server. Clients use standard internet protocols, such as HTTP (Hypertext Transfer Protocol) or HTTPS (HTTP Secure), to connect and communicate with the database server.
- *Security:* Due to the open nature of the Internet, security is a paramount concern for Internet databases. They implement robust authentication and authorization mechanisms to control access to data and prevent unauthorized users from tampering with sensitive information.
- *Scalability:* Internet databases often need to handle a large number of concurrent users and data requests. Scalability is a critical requirement, and Internet database systems are designed to scale both vertically (adding more resources to a single server) and horizontally (distributing data across multiple servers).
- *Data Storage:* Internet databases can use various storage options, including relational databases, NoSQL databases, and cloud-based data storage solutions. The choice of database technology depends on the nature of the data and the specific requirements of the application.
- *Data Replication and High Availability:* To ensure high availability and fault tolerance, internet databases may employ data replication techniques. Data is copied across multiple servers in different

geographical locations to handle hardware failures and reduce response time for users in different regions.

- *Data Privacy and Compliance:* Internet databases must adhere to data privacy regulations and compliance standards, such as GDPR (General Data Protection Regulation) and HIPAA (Health Insurance Portability and Accountability Act). They should handle user data responsibly and provide options for users to manage their data privacy preferences.
- *Caching:* Internet databases often use caching mechanisms to reduce the load on the database server and improve the response time for frequently accessed data. Caching stores frequently requested data in memory, allowing quicker access without hitting the database for every request.

8.2 World Wide Web

The World Wide Web (WWW or Web) is a global information system that allows users to access and interact with a vast collection of resources and information over the Internet. It is a network of interconnected documents and other resources, linked by hyperlinks and URLs (Uniform Resource Locators). The Web is a crucial part of modern society and plays a significant role in how we access and share information, conduct business, and communicate with others.

History of World Wide Web (WWW):

The World Wide Web (WWW) was invented by British computer scientist Sir Tim Berners-Lee in 1989 while working at CERN. He proposed the idea of a distributed hypertext system to share information over the Internet. In 1990, Berners-Lee developed the first web server and web browser, introducing HTTP and HTML as the foundation of the Web. The technology was released into the public domain in 1993, leading to the rapid growth of websites and the launch of commercial web browsers like Netscape Navigator and Internet Explorer in the mid-1990s. The dot-com bubble saw significant investments in internet startups, followed by the emergence of Web 2.0, social media, and the standardization of web technologies by the World Wide Web Consortium (W3C) in the early 2000s. Today, the World Wide Web continues to evolve, shaping the way we access information and interact with each other globally.

Components of the World Wide Web:

- *Web Architecture:* The Web is built upon a client-server architecture. Web clients, also known as web browsers, are used by individuals to access and view web pages. Web servers, on the other hand, store and deliver web pages and other resources in response to client requests. DBMS can act as a web server, providing access to data stored in databases via the Internet.
- *Web Pages and Content:* Web pages are the basic building blocks of the Web. They are written in HTML (Hypertext Markup Language) and can include multimedia elements like images, videos, and audio. A

web page can be a static document or dynamically generated content based on user input or data from a DBMS.

- *URL and Hyperlinks:* Uniform Resource Locator (URL) is the addressing system used on the Web to identify resources, such as web pages or files. URLs consist of various components, including the protocol (e.g., HTTP, HTTPS), the domain name, and the specific path to the resource. Hyperlinks, also known as links, are connections between different web pages and resources. They allow users to navigate the web by clicking on text or images, leading to other relevant pages or resources.
- *Web Applications and DBMS:* Web applications are software programs that run on web servers and are accessed through web browsers. These applications can utilize DBMS to store, retrieve, and manage data. For example, e-commerce websites, social media platforms, and online banking applications rely heavily on DBMS to store user profiles, product catalogs, transaction records, etc.
- *Database-Driven Websites:* Database-driven websites are web applications that use DBMS to dynamically generate web pages and content based on data from the database. When a user requests specific information, the web server retrieves the relevant data from the database, processes it, and generates a web page containing the requested information.
- *Database Connectivity:* To interact with a database, web applications use various programming languages and technologies like PHP,

Python, Java, ASP.NET, etc. These programming languages provide libraries and APIs (Application Programming Interfaces) to connect to the DBMS, execute queries, and process the retrieved data.

- *Security Considerations:* Using DBMS on the World Wide Web brings security challenges. Web applications must employ secure coding practices to prevent common vulnerabilities like SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF). Additionally, DBMS itself should be adequately secured with strong authentication and access control mechanisms.
- *Scalability:* As web traffic and user demands increase, the DBMS and the web server must be scalable to handle the load. This can involve load balancing, sharding, caching, and other techniques to ensure the system remains responsive and available.

Working of World Wide Web:

The World Wide Web (WWW) is a global information system that revolutionized how we access, share, and interact with vast amounts of information over the Internet. It involves a complex interplay of components and protocols that facilitate the seamless exchange of data between web servers and web browsers. When a user enters a Uniform Resource Locator (URL) or clicks on a hyperlink, the web browser sends a request to the corresponding web server using the Hypertext Transfer Protocol (HTTP). The web server processes the request, locates the requested resource, and sends it back to the web browser as an HTTP response. Web pages, which are the

basic building blocks of the web, are written in Hypertext Markup Language (HTML) and may contain multimedia elements like images and videos. Web servers can serve static web pages directly from their storage, but they can also generate dynamic content by processing user input or retrieving data from databases using server-side scripting languages like PHP or Python. Hyperlinks within web pages allow users to navigate between different resources and websites. Once the web browser receives the response, it interprets the HTML, displays the content, and renders the web page on the user's device. This process, together with the stateless nature of HTTP, enables users to interact with an ever-expanding array of web-based applications, services, and resources available on the World Wide Web.

Features of the World Wide Web:

The World Wide Web (WWW) offers a wide range of features that have transformed the way we access and interact with information on the Internet.

- *Hyperlinks:* Hyperlinks are connections between web pages and resources. They allow users to navigate seamlessly between different web pages and websites by clicking on text or images that direct them to other relevant content.

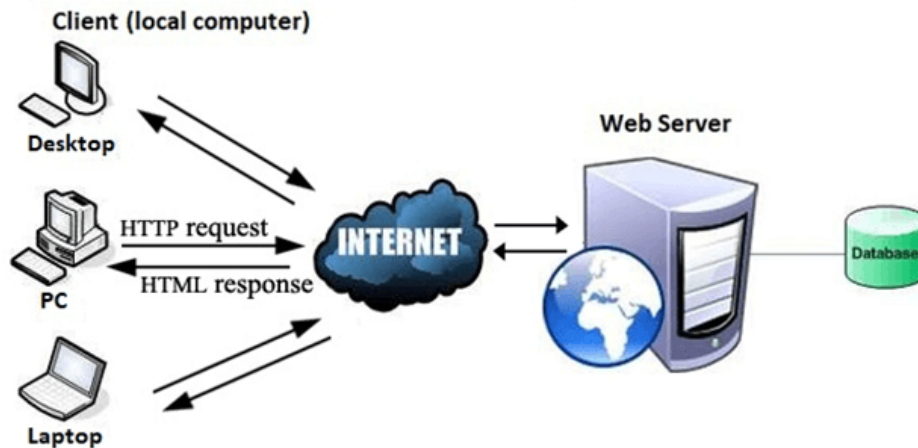


Fig – World Wide Web

- *Uniform Resource Locator (URL)*: URLs are web addresses used to identify and locate specific resources on the Internet. Users input URLs into web browsers to request resources hosted on web servers.
- *Hypertext Markup Language (HTML)*: HTML is the standard markup language used to create web pages. It enables the structuring and formatting of content, defining links, and embedding multimedia elements like images, videos, and audio.
- *Hypertext Transfer Protocol (HTTP/HTTPS)*: HTTP is the fundamental protocol for communication on the WWW. It governs how web clients (browsers) and web servers interact, allowing the retrieval of web resources upon user requests. HTTPS provides secure communication using encryption.
- *Web Browsers*: Web browsers, such as Chrome, Firefox, Safari, and Edge, provide user interfaces to access and display web pages. They

interpret HTML, CSS, and JavaScript code to render web content correctly.

- *Multimedia Content:* The WWW supports various multimedia elements, including images, videos, audio, and interactive media, allowing for engaging and dynamic user experiences.
- *Search Engines:* Search engines like Google, Bing, and Yahoo index and organize vast amounts of web content, enabling users to search for specific information and resources quickly.
- *Web Standards and Protocols:* The WWW adheres to a set of standards and protocols defined by organizations like the World Wide Web Consortium (W3C) and the Internet Engineering Task Force (IETF) to ensure compatibility, interoperability, and uniformity across the web.
- *Cookies and Sessions:* Cookies are small pieces of data stored on the user's device by web servers, allowing websites to remember user preferences and maintain state between requests. Sessions enable web applications to maintain user-specific data during the user's visit.
- *E-commerce and Online Transactions:* The WWW facilitates online shopping and secure financial transactions through e-commerce websites, enabling users to purchase goods and services from the comfort of their homes.
- *Social Networking:* Social media platforms are an integral part of the WWW, allowing users to connect, share content, and communicate with others across the globe.

- *Web Applications:* Web applications run on web servers and provide interactive user experiences. They can be simple, like webmail clients, or complex, like online banking systems, delivering various services over the web.

Difference Between World Wide Web and the Internet:

World Wide Web (WWW)	Internet
A subset of the Internet.	The global network infrastructure connects computers and networks worldwide.
An information-sharing system that operates on top of the Internet.	Encompasses a wide range of applications and services, including the World Wide Web.
Consists of interconnected documents and resources accessed through web browsers.	Allows diverse data exchange, communication, and networking.
Uses URLs and hyperlinks to navigate between web pages.	Comprises physical components like routers, cables, and wireless connections.
Invented by Tim Berners-Lee in 1989.	Development began in the late 1960s with ARPANET.

8.3 Client Side Scripting and Applets

Client-Side Scripting:

Client-side scripting refers to the process of writing and executing scripts on the user's web browser, usually using languages like JavaScript, HTML, and CSS. These scripts are embedded directly into the web page's HTML code and are executed on the client-side, meaning they run on the user's computer rather than on the web server. Client-side scripting enables web developers to enhance the interactivity and functionality of web pages without the need for server-side processing.

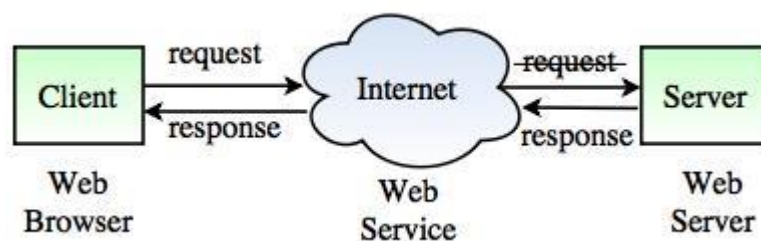


Fig – Client Server Architecture

Some Characteristics of client-side scripting:

- *Execution on the User's Browser:* The scripts are sent to the user's web browser along with the HTML code and are executed there. This reduces the load on the web server and enhances the responsiveness of web pages.
- *Interactivity:* Client-side scripts enable the creation of interactive elements on web pages, such as form validation, animations, image sliders, and pop-up dialogs. These features enhance the user experience and engagement.

- *Real-time Validation:* Client-side scripts can validate user input in real-time, providing immediate feedback on form submissions, such as checking for valid email addresses or password strength.
- *Manipulating DOM:* Client-side scripts have access to the Document Object Model (DOM), which represents the structure of the web page. Developers can use DOM manipulation to dynamically change content and styles, creating dynamic and responsive user interfaces.
- *Faster Response:* Since the scripts are executed locally on the user's browser, the response time for user interactions is generally faster, leading to a smoother user experience.

Advantages of Client-side scripting:

Client-side scripting offers several advantages that contribute to the development of dynamic and interactive web applications.

- *Improved User Experience:* Client-side scripting enables developers to create interactive and responsive user interfaces, enhancing the overall user experience. Actions like form validation, animations, and real-time updates provide immediate feedback to users, making web pages more engaging and user-friendly.
- *Reduced Server Load:* Since client-side scripts run on the user's web browser, they offload processing tasks from the web server. This reduces the burden on the server and contributes to faster response times and improved performance, especially during peak traffic periods.

- *Real-Time Validation:* Client-side scripts can perform real-time validation of user input, ensuring that data is entered correctly before it is submitted to the server. This immediate feedback saves time and prevents unnecessary server requests for data validation.
- *Faster Response Times:* By handling tasks locally on the user's computer, client-side scripts reduce the need for round-trips to the server. This results in faster load times for web pages and a smoother user experience, especially on low-bandwidth or slow internet connections.
- *Enhanced Interactivity:* Client-side scripting allows developers to add interactivity and dynamism to web pages, creating a more interactive and engaging environment for users. Elements like image sliders, pop-up dialogs, and dynamic content updating enrich the user experience.
- *Reduced Bandwidth Usage:* Client-side scripting can minimize the amount of data exchanged between the client and server. For example, AJAX (Asynchronous JavaScript and XML) enables partial page updates without reloading the entire page, reducing the amount of data transferred over the network.
- *Device Independence:* Since client-side scripts run on the user's web browser, they work independently of the user's device or operating system. This makes web applications more accessible and compatible with various devices, including desktops, laptops, tablets, and smartphones.

- *No Server-Side Dependency:* Client-side scripts can function independently of the server, which allows for offline functionality and offline caching of web pages. Users can still interact with certain parts of the application even when they are not connected to the Internet.
- *Easier Debugging:* Debugging client-side scripts is generally easier because developers can use browser developer tools to inspect, analyze, and test scripts directly within the user's browser.

Disadvantages of Client-side scripting:

While client-side scripting offers several advantages, it also comes with some disadvantages and challenges.

- *Security Risks:* Client-side scripts run on the user's web browser, which means they can be accessed and potentially modified by users. This makes them susceptible to security risks, such as code injection attacks or cross-site scripting (XSS) vulnerabilities if not properly secured.
- *Browser Compatibility:* Different web browsers may interpret client-side scripts differently or support different features, leading to inconsistencies in the rendering and behavior of web pages. Web developers often need to test and adjust scripts to ensure compatibility across various browsers.
- *Code Exposure:* Client-side scripts are visible to users, and all the code is sent to the user's browser. This exposes the code logic and

implementation, making it easier for potential attackers to reverse engineer or exploit the application's vulnerabilities.

- *Performance Impact:* Complex client-side scripts can increase the loading time of web pages, especially on older devices or slower internet connections. Heavy use of client-side scripts can also consume significant memory and processing power on the user's device.
- *Limited Access to Server Resources:* Client-side scripts have restricted access to server resources due to security reasons. This limitation can hinder certain functionalities that require access to sensitive server-side data or system resources.
- *Dependency on User's Browser:* The performance and behavior of client-side scripts are dependent on the user's web browser and its capabilities. If a user disables JavaScript or uses an older browser version, certain functionalities may not work as expected.
- *Lack of Offline Functionality:* Client-side scripts typically require an active internet connection to function correctly. This limitation means that some web applications may not be accessible or fully functional offline.
- *Difficulty in Managing Complex Logic:* While client-side scripts can handle front-end logic, more complex business logic or data processing may be challenging to manage on the client side alone. Server-side scripting is often required to handle critical data operations and ensure data integrity.

- *Code Repetition:* If complex operations are performed on both the client-side and server side, there may be a need to duplicate code for consistency, leading to code maintenance challenges and potential code discrepancies.

Applets:

An applet is a small, self-contained program written in the Java programming language. It is designed to run within a web browser and is executed on the client-side (user's computer). Applets were popular in the early days of the web for adding interactive and multimedia content to web pages.

Characteristics of applets:

- *Java Technology:* Applets are written in Java, a platform-independent programming language. This means that applets can run on any operating system and web browser that supports Java.
- *Integration with Web Pages:* Applets are embedded directly into HTML code on web pages using the <applet> tag or the <object> tag.
- *Restricted Environment:* Applets run in a restricted environment called a "sandbox," which prevents them from accessing the user's local file system or other sensitive resources on their computer. This ensures a level of security for users when running applets from untrusted sources.

- *Interactivity and Multimedia:* Applets were primarily used to add interactive elements and multimedia content to web pages. They could create animations, games, interactive forms, 3D graphics, and more.
- *Platform Independence:* As long as the user's web browser supports Java and has the Java Runtime Environment (JRE) installed, applets can run on any platform, including Windows, macOS, Linux, and others.
- *Decreased Popularity:* While applets were popular in the past, they have largely fallen out of use due to security concerns, performance issues, and the emergence of more modern web technologies like HTML5, CSS, and JavaScript.
- *Need for Java Plugin:* To run applets, users must have the Java Runtime Environment (JRE) installed on their computers. However, as Java applets have become less common, many modern web browsers have dropped support for Java plugins.

Lifecycle of Applets:

In the context of Database Management Systems (DBMS), the life cycle of applets remains the same as in general Java applets. However, when using applets in conjunction with DBMS, there are additional considerations for integrating applets with the database

1. *Initialization:* The `init()` method is called when the applet is loaded onto a web page. During the initialization phase, you can set up the applet's graphical user interface (GUI) and establish connections with the database server.

2. *Starting and Running:* The start() method is called after initialization and whenever the applet is started or reactivated. In this phase, you can initiate any processes or data retrieval tasks from the database. For example, you may fetch initial data to display on the applet's GUI.
3. *Interacting with Database:* Once the applet is running, users can interact with its GUI elements. These interactions can trigger various actions, such as querying the database for specific data or submitting data to be stored in the database.

Applets can use technologies like JDBC (Java Database Connectivity) to connect to the database server, execute SQL queries, and retrieve or modify data.
4. *Stopping:* The stop() method is called when the applet is about to be deactivated or when the user navigates away from the web page containing the applet. In this phase, you may pause or terminate any ongoing database-related tasks.
5. *Destruction:* The destroy() method is called when the applet is about to be unloaded or when the web page is closed. This is the opportunity to clean up resources, release database connections, and perform any necessary cleanup related to the database operations.

In modern web development, applets have become less common due to security concerns and the availability of other technologies like JavaScript and HTML5. Instead of using Java applets, web developers often rely on

other approaches, such as using web APIs (e.g., RESTful APIs) to interact with the database directly from client-side JavaScript or using server-side scripting languages (e.g., PHP, Python) to handle interactions with the database on the web server.

Uses of Applets:

- *Data Visualization:* Applets could be used to create interactive data visualization components, such as charts, graphs, and maps, to present database information in a user-friendly and visually appealing manner.
- *Interactive Querying:* Applets could serve as front-end interfaces for users to interactively query the database. Users could input search criteria, and the applet would submit SQL queries to the database server and display the results.
- *CRUD Operations:* Applets could provide a user interface for Create, Read, Update, and Delete (CRUD) operations on the database. Users could use the applet to add new records, view existing data, modify records, and delete data from the database.
- *Form-Based Data Entry:* Applets could be used to create form-based data entry interfaces, allowing users to input data through the applet's GUI and store it in the database.
- *Real-Time Data Updates:* Applets could facilitate real-time updates of data from the database, displaying changes to users as they occur without requiring a full page refresh.

- *Custom Reports:* Applets could generate custom reports based on data retrieved from the database, presenting the information in a format tailored to the user's needs.
- *Database Administration:* Applets could serve as simple tools for database administrators to perform tasks like database backups, user management, and configuration changes.

8.4 Web Servers and Sessions

Web Servers:

Web servers are software applications or computer systems that handle HTTP (Hypertext Transfer Protocol) requests from clients (typically web browsers) and serve web content over the internet. They play a fundamental role in hosting websites and delivering web pages, images, videos, and other resources to users who access those websites through their browsers.

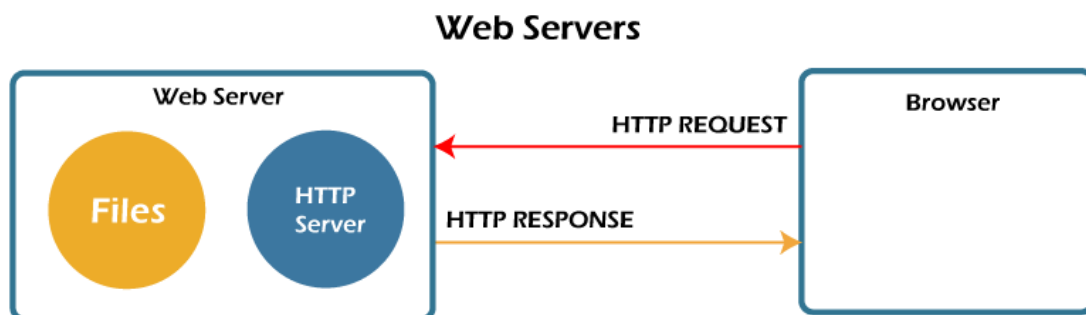


Fig – Web servers

Web servers are the backbone of the internet, enabling websites to function and deliver content to users worldwide. When a user interacts with a website, the web server plays a crucial role in handling the request and delivering the

requested content. The process begins when a user enters a URL or clicks a link in their web browser, which triggers an HTTP request to the web server. The web server receives the request, analyses the URL to identify the requested resource, and searches for it on its file system or in its cache. If the resource is a static file, like an HTML page or an image, the web server directly delivers it to the user's browser as an HTTP response. For dynamic content, the web server communicates with an application server or server-side scripting engine to generate the content based on user input or database data.

Once the appropriate content is ready, the web server constructs an HTTP response, including the requested content and metadata, and sends it back to the user's browser. The browser then renders the web page or displays the content to the user. Web servers also play a role in security, using SSL/TLS encryption for secure communication (HTTPS) and implementing various measures to protect websites and user data. The process is stateless, but web applications use techniques like cookies or URL parameters to maintain user sessions and store stateful information. Load balancing allows multiple web servers to work together, distributing requests evenly to improve performance and reliability. In summary, web servers act as intermediaries, facilitating communication between users and web applications, ensuring the smooth delivery of web content, and providing essential security and performance features.

Features and functionalities of a web server:

- *HTTP Request Handling:* A web server is capable of receiving and processing HTTP requests from clients. These requests can include a specific URL (Uniform Resource Locator) or the name of a file or resource the client wants to access.
- *Content Delivery:* The web server stores the web content, including static files like HTML, CSS, JavaScript, images, videos, and downloadable files. When a client requests a resource, the web server locates and sends the requested content in the form of an HTTP response.
- *Static vs. Dynamic Content:* Web servers can deliver both static and dynamic content. Static content remains unchanged unless manually modified, whereas dynamic content is generated on the fly in response to user input or other external factors. For dynamic content, web servers can communicate with application servers or server-side scripts to generate the appropriate responses.
- *MIME Types:* Web servers use MIME (Multipurpose Internet Mail Extensions) types to specify the nature of the content being delivered. These types indicate whether the content is HTML, CSS, JavaScript, image, audio, video, or other types of data.
- *Virtual Hosting:* Web servers can handle multiple websites (domains) on the same physical server through virtual hosting. This allows different websites to share the same server resources while appearing as separate entities to visitors.

- *Security:* Web servers implement various security measures to protect websites and user data. This includes SSL/TLS encryption for secure communication (HTTPS), access control mechanisms, and protection against common web vulnerabilities.
- *Authentication and Authorization:* Web servers can handle user authentication and authorization, ensuring that only authorized users can access certain parts of a website or perform specific actions.
- *Load Balancing:* In high-traffic scenarios, web servers can distribute incoming requests across multiple servers (load balancing) to improve performance and ensure high availability.
- *Logging and Monitoring:* Web servers maintain logs of incoming requests and responses, providing valuable information for troubleshooting, monitoring server performance, and analyzing website traffic.
- *Reverse Proxy:* Some web servers can act as reverse proxies, forwarding requests to backend application servers or other web servers. This helps improve performance, security, and load distribution.
- *Compression:* Web servers can compress web content before sending it to clients, reducing the amount of data transferred and improving page load times.

There are several web servers available, each with its own set of features, performance characteristics, and popularity.

1. Apache HTTP Server (Apache):

- Apache is one of the most popular and widely used web servers globally.
- It is open-source and free to use, available on various operating systems.
- Apache is highly configurable and extensible through modules, allowing it to support a wide range of functionalities.

2. Nginx:

- Nginx (pronounced "engine X") is known for its high performance and scalability.
- It is lightweight and efficient in handling concurrent connections, making it suitable for serving static content and acting as a reverse proxy.

3. Microsoft Internet Information Services (IIS):

- IIS is developed by Microsoft and designed to run on Windows Server operating systems.
- It is commonly used for hosting websites and ASP.NET web applications.

4. LiteSpeed Web Server:

- LiteSpeed is known for its high-performance capabilities and is designed to be a drop-in replacement for Apache.
- It provides various features, including page caching, ESI (Edge Side Includes), and advanced security options.

5. Tomcat (Apache Tomcat):

- Tomcat is a lightweight application server and web server designed to run Java Servlets and JavaServer Pages (JSP).
- It is commonly used for deploying Java web applications.

6. IBM HTTP Server (IBM HTTPD):

- IBM HTTP Server is based on Apache and is included in IBM's WebSphere Application Server and WebSphere Portal offerings.
- It is commonly used in enterprise environments for hosting web applications.

Web Sessions:

Web sessions, also known as user sessions or session management, refer to the mechanism used to maintain stateful communication between a web server and a web client (usually a web browser) over a series of HTTP requests and responses. Web sessions enable web applications to recognize and remember individual users, store user-specific data, and maintain user state during their interactions with the website.

How Web Sessions Work:

- *Session Initialization:* When a user visits a website for the first time, the web server creates a unique session for that user. A session ID is generated and sent to the user's web browser, usually as a cookie.

- *Session Tracking:* Subsequent requests from the same user contain the session ID in the HTTP request. The web server uses the session ID to identify the associated session and retrieve any stored session data.
- *Data Persistence:* During the session, web applications can store user-specific information on the server-side using the session ID as the key. This data persists across multiple requests and allows the application to remember user preferences, settings, and activities.
- *Expiry and Termination:* Web sessions can have an expiration time, after which they are considered inactive. When a session expires (usually after a period of inactivity), the session data may be deleted from the server. Users can also explicitly terminate their sessions, which clears session data.

Uses of Web Sessions:

- *User Authentication:* Web sessions are commonly used to handle user authentication. After a user logs in, the session keeps track of the authenticated state, allowing users to access restricted areas of the website without re-authenticating for each request.
- *Personalization:* Web sessions enable websites to provide personalized experiences to users based on their interactions and preferences during the session. This can include remembering language preferences, shopping cart items, or user-specific settings.
- *Maintaining Stateful Interactions:* Some web applications require users to engage in multi-step interactions or transactions. Web sessions

allow the application to preserve the state of these interactions across multiple pages, ensuring a seamless user experience.

- *Enhancing Performance:* Web sessions reduce the need to repeatedly authenticate and manage user data on each request, which can improve the performance and responsiveness of web applications.
- *Security:* Web sessions offer a more secure way to handle user states than passing sensitive information via URLs or hidden form fields, as session data is stored server-side and is not accessible to users.
- *E-commerce and Shopping Carts:* In e-commerce applications, web sessions are used to maintain shopping cart contents and keep track of user selections until the purchase is completed.

8.5 Web Services

Web services are software components or applications that allow different systems and platforms to communicate and exchange data over the Internet. They provide a standardized way for applications to interact with each other, regardless of the programming languages, operating systems, or platforms they are built on. Web services enable seamless integration and interoperability between diverse systems, making it easier for developers to build distributed and interconnected applications.

How do web Services work:

Web services enable different software applications to communicate and exchange data over the Internet in a standardized and interoperable manner. They follow a client-server model, where one application acts as the service

provider (server), offering specific functionalities, and the other application serves as the client, consuming or using the service. The process begins with the service provider defining the functionalities and data the service offers, usually using XML or JSON to describe the service's operations. The provider publishes the service on the internet, making it available for discovery by potential clients through public directories or service registries. Clients looking for a specific service can locate it based on its description or name. To use a web service, a client sends a request to the service provider, containing the necessary data or parameters. The web service processes the request, performs the specified operation, and generates a response, usually in XML or JSON format, containing the requested data or results. The client receives the response and uses the data for its purposes. Web services use standard communication protocols like HTTP or SOAP to send requests and responses, making them platform-independent and allowing seamless integration between diverse systems.

Features of Web Services:

- *Platform-Independent:* Web services enable communication between applications built on different platforms and programming languages, making them platform-independent and highly flexible.
- *Interoperable:* Web services use standard communication protocols like HTTP and standard data formats like XML or JSON, ensuring seamless integration and interoperability between different systems.

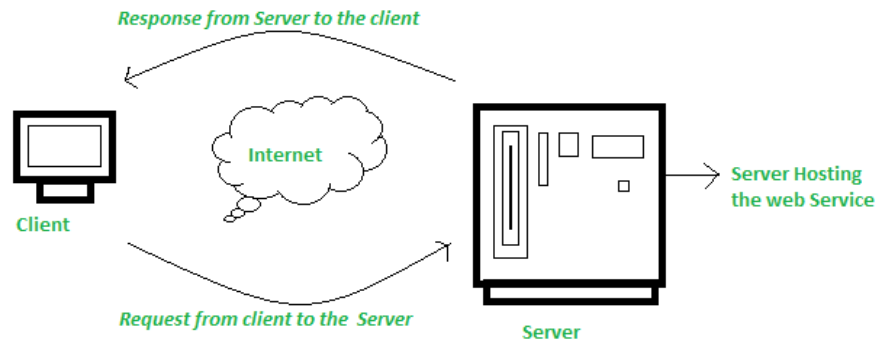


Fig – Web services

- *Loosely Coupled:* Web services operate independently of each other, and their interactions are loosely coupled, meaning that changes in one service do not necessarily affect other services.
- *Reusable:* Web services can be used and accessed by multiple applications, promoting code reusability and simplifying development efforts.
- *Secure:* Web services can be secured using various authentication and encryption mechanisms, ensuring that data exchanged between systems remains confidential and protected from unauthorized access.

Functions of Web Services:

- Enable communication and data exchange between different software applications over the internet.
- Provide a standardized and platform-independent way for applications to interact and share information.

- Facilitate seamless integration and interoperability between diverse systems and platforms.
- Promote code reusability by allowing multiple applications to access and use the same web service.
- Support loose coupling between services, ensuring that changes in one service do not affect others.
- Scale to handle a large number of concurrent requests, making them suitable for high-traffic applications.
- Enhance security with authentication and encryption mechanisms to protect data during transmission.
- Offer cost-effective solutions by leveraging existing functionalities and services instead of building from scratch.
- Facilitate real-time communication and dynamic content generation for interactive applications.
- Improve service discovery through public directories or service registries, making them easily accessible to potential consumers.
- Enable remote access to functionalities and data hosted on remote servers, supporting distributed systems. Allow data exchange in standardized formats like XML or JSON, making it easier to share and interpret data between applications.

Advantages of Web Services:

Web services offer numerous advantages that make them a powerful and popular technology for integrating different systems and enabling communication between applications.

- *Platform Independence:* Web services use standard communication protocols like HTTP and data formats like XML or JSON, making them platform-independent. They can be accessed and used by applications built on different programming languages and operating systems.
- *Interoperability:* Web services promote interoperability by providing a standardized way for applications to communicate and exchange data. This enables seamless integration between diverse systems, regardless of their technological differences.
- *Reusability:* Web services are designed to be reusable components that can be accessed and used by multiple applications. This promotes code reusability and simplifies development efforts.
- *Loose Coupling:* Web services operate independently of each other, and their interactions are loosely coupled. Changes in one service do not necessarily impact other services, ensuring flexibility and modularity in the system.
- *Scalability:* Web services can handle a large number of concurrent requests, making them scalable to meet the demands of growing applications and user bases.
- *Security:* Web services can be secured using various authentication and encryption mechanisms, ensuring the privacy and integrity of data exchanged between systems.

- **Cost-Effectiveness:** Web services enable businesses to leverage existing functionalities and services without the need to build everything from scratch. This can lead to cost savings and faster development cycles.
- *Wide Adoption:* Web services are widely adopted and supported by various platforms, making them a reliable and well-established technology for integration and communication.
- *Data Exchange Flexibility:* Web services can exchange data in a standardized format like XML or JSON, making it easier to share and interpret data between applications.
- **Remote Access:** Web services allow applications to access functionalities or data hosted on remote servers, enabling distributed and decentralized systems.
- *Real-time Communication:* Web services can offer real-time communication and dynamic content generation, making them suitable for interactive and responsive applications.

8.6 Server-Side Scripting

Server-side scripting is a programming technique where scripts or code are executed on the web server during the processing of a client's request. Unlike client-side scripting, which runs on the user's web browser, server-side scripting allows web applications to handle data processing, database interactions, and other operations on the server before delivering the final result to the client's browser. The server-side scripts are typically written in

languages such as PHP, Python, Ruby, Java, or ASP.NET, and they enable dynamic content generation, database querying, user authentication, and complex business logic, among other functionalities. Server-side scripting enhances security by keeping sensitive operations and data processing on the server while providing platform independence and consistent behaviour across different client devices and browsers.

Advantages of Server-Side Scripting:

- *Data Security:* Server-side scripts keep sensitive data and business logic on the server, preventing direct access and manipulation by clients, thus enhancing security.
- *Platform Independence:* Server-side scripts run on the server, making them independent of the client's web browser or device, ensuring consistent behavior across different platforms.
- *Performance:* Server-side scripts offload resource-intensive tasks from the client's device, reducing the load on the client's hardware and improving overall application performance.
- *Code Protection:* Server-side scripts are not directly visible to users, keeping the underlying code and algorithms hidden, protecting intellectual property, and preventing unauthorized modifications.
- *Database Interaction:* Server-side scripts can interact directly with databases, allowing efficient data retrieval, storage, and manipulation, enabling robust data-driven web applications.

Disadvantages of Server-Side Scripting:

- *Server Load:* Server-side scripting requires server resources to process and execute scripts, leading to increased server load, which can impact application performance during high traffic.
- *Reduced Responsiveness:* Server-side processing may introduce latency as data is processed on the server before being delivered to the client, resulting in reduced responsiveness for real-time interactions.
- *Limited Client-Side Control:* Since most processing occurs on the server, client-side control is limited, which may result in a less interactive user experience for some applications.
- *Scalability Challenges:* As the number of users and requests increases, server-side processing may become a scalability challenge, necessitating load balancing, and resource management.
- *Code Complexity:* Server-side scripting can lead to more complex code and require additional maintenance and debugging efforts, especially in large-scale applications.

8.7 Summary

Internet databases are fundamental components that underpin the dynamic and interactive nature of the World Wide Web. They store vast amounts of information, making it readily accessible to users across the globe. The World Wide Web (WWW) acts as the interface for users to access this wealth of data through web browsers. It allows users to navigate through web pages, click on links, and interact with various web-based applications.

Client-side scripting and applets are key technologies that enhance the user experience by enabling dynamic content generation directly within the user's web browser. Client-side scripts, usually written in JavaScript, enable real-time interactions, form validation, and dynamic content updates without the need to reload the entire web page. Applets, on the other hand, are small Java programs that run within a web browser, adding interactivity and multimedia capabilities to web pages.

Web servers and sessions are vital for maintaining stateful communication between the client and server. Web servers process HTTP requests from web browsers, fetching web pages and resources and delivering them back to users as HTTP responses. Sessions allow web servers to recognize and remember individual users, storing user-specific data on the server-side. This enables websites to personalize user experiences, handle authentication, and manage user interactions across multiple requests.

Web services are essential for integrating disparate systems and sharing data across the internet. They enable communication between applications running on different platforms and programming languages. Web services typically use standard protocols like XML and JSON to exchange data in a platform-independent manner.

Server-side scripting empowers web applications to process data on the server before delivering the final result to users. Server-side scripts can interact with databases, perform complex calculations, and handle business

logic. Popular server-side scripting languages include PHP, Python, Ruby, and Java.

8.8 Review Questions

- 1 What are Internet databases, and how do they differ from traditional databases?
- 2 Explain the role of the World Wide Web in the context of Internet databases.
- 3 Describe the concept of client-side scripting and its significance in web-based applications.
- 4 How do applets differ from regular client-side scripts, and what purposes do they serve?
- 5 What is the role of web servers in facilitating communication between clients and databases?
- 6 What are sessions in the context of Internet databases, and how do they maintain state across web requests?
- 7 List and briefly explain some common web services that can interact with Internet databases.
- 8 Differentiate between client-side scripting and server-side scripting in web development.
- 9 How can server-side scripting languages interact with databases to handle dynamic content generation?
- 10 Explain the importance of security measures in Internet databases and web applications.

8.9 Keywords

- *Internet Databases:* Databases accessible over the internet that store and manage vast amounts of information, enabling web applications to access, retrieve, and manipulate data dynamically.
- *World Wide Web (WWW):* The interconnected network of websites and web pages that are accessible over the internet, allowing users to navigate, interact, and retrieve information using web browsers.
- *Client Side Scripting:* A scripting technique where scripts (usually written in JavaScript) run on the client's web browser, enabling dynamic content generation, form validation, and real-time interactions without the need for server-side processing.
- *Applets:* Small Java programs that run within a web browser, providing interactivity and multimedia capabilities to web pages. Applets were commonly used in the past but have become less popular due to security concerns and the availability of other web technologies.
- *Web Servers:* Software applications or computer systems that handle HTTP requests from clients (web browsers) and serve web content over the internet, ensuring the delivery of web pages and resources to users.
- *Sessions:* Mechanisms used to maintain stateful communication between web servers and clients, allowing web applications to recognize and remember individual users, store user-specific data, and provide personalized experiences.

- *Services:* Web services are technologies that enable communication between different systems and platforms over the Internet. They facilitate data exchange and interoperability between web applications using standard protocols like XML and JSON.
- *Server Side Scripting:* A scripting technique where scripts (e.g., PHP, Python, Ruby) run on the web server, processing data and performing complex operations before delivering the final result to users' web browsers. Server-side scripting enables interaction with databases and handling of business logic on the server.
- *HTTP:* Hypertext Transfer Protocol, the foundation of data communication for the World Wide Web. It defines how messages are formatted and transmitted, and how web servers and browsers should respond to various commands.
- *Web Browser:* Software used by users to access and view websites on the World Wide Web. Popular web browsers include Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.
- *Dynamic Content:* Web content that can change in real time based on user interactions, data input, or other external factors. Client-side scripting and server-side scripting enable the creation of dynamic content on websites.
- *URL:* Uniform Resource Locator, a web address used to locate specific resources on the internet. URLs define the protocol (e.g., HTTP, HTTPS), the domain name, and the path to the resource.

- *HTML*: Hypertext Markup Language, the standard markup language used to create web pages. It defines the structure and layout of web content, allowing browsers to interpret and display the content correctly.
- *CSS*: Cascading Style Sheets, a style sheet language used to control the presentation and layout of HTML documents. CSS enables web designers to apply styles and formatting to web pages.
- *SSL/TLS*: Secure Socket Layer (SSL) and Transport Layer Security (TLS) protocols, are used to encrypt data transmitted between web servers and browsers, providing secure and private communication over the internet (HTTPS).

8.10 References

1. Date C J, “ An Introduction to Database Systems”, Addison Wesley
2. Korth, Silbertz, Sudarshan,” Database Concepts”, McGraw Hill
3. Elmasri, Navathe, “Fundamentals of Database Systems”, Addison Wesley
4. Leon & Leon,”Database Management Systems”, Vikas Publishing House
5. Bipin C. Desai, “ An Introduction to Database Systems”, Gagotia Publications
6. Majumdar & Bhattacharya, “Database Management System”, TMH